Senior Design — Lab 8

October 28, 2026

- From Elevator Pitch to Technical Design

- Presentation 1 Feedback • Presentation 2 • Demo 0

Today’s Agenda

- Presentation 1: what we learned
- Improve your project storyline
- Presentation 2: technical requirements and rubric
- Demo 0: individual code review
- Sprint planning and course reminders

Learning Goals

- Tell a clear, coherent project story
- Explain a technical architecture and why it works
- Understand Presentation 2 evaluation criteria
- Prepare evidence of your individual technical work
- Define actionable next-sprint goals

Upcoming Milestones

- Sunday, Nov 1 — Teamwork Survey (individual, confidential, ungraded)
- Week of Nov 3 — Demo 0 individual progress review
- Wednesday, Nov 4 — Presentation 2 (7 minutes/team)
- Nov 11 — Alumni Panel
- Nov 15 — Writing 3 (team submission)
- Dec 2 — Presentation 3; Week of Dec 8 — Alpha Review

Resources: Course schedule

Presentation 1 — Class Results

- Speaking: 8.60 / 10
- Position: 8.87 / 10
- Engagement: 8.28 / 10 (lowest average)
- Slides: 8.83 / 10
- Overall: 86.44 / 100
- 55 students graded independently by both instructors

What Worked — What to Improve

- Strengths: promising projects, preparation, and technical understanding
- Improve: audience engagement and eye contact
- Avoid reading from phones or slides
- Make team transitions smoother
- Explain technical concepts in accessible language
- Most importantly: strengthen your project storyline

Storytelling: Technologies Are Not the Story

- Weak: “We use Python, React, PostgreSQL, and an API.”
- Stronger: “Our users face X; we build Y; our system addresses Z.”
- Problem → people → solution → design → impact
- Explain *why* the technology matters, not just *which* tools you chose

Five-Step Project Storyline

- 1. Problem: what are you solving?
- 2. Users: who benefits and why?
- 3. Solution: what does your system do?
- 4. Design: how will it work technically?
- 5. Impact: what does success look like?
- For Presentation 2, give the most attention to step 4.

Storytelling Example — Before and After

- Weak: “Our team uses React, Flask, ML, and a calendar UI.”

- Improved: “Students struggle to prioritize deadlines and study time. We propose a scheduling assistant that uses availability and assignment data to recommend plans.”
- Technical challenge: adapt recommendations when constraints change.

- What changed? The second version establishes purpose and design motivation.

Think–Pair–Share: Rewrite Your Story

6-minute activity

- 2 min — Write 2 sentences explaining your project without technology names
- 2 min — Explain it to a student outside your team
- 2 min — Ask your partner to restate the problem and solution
- Discuss: what was confusing or missing?

Better Slides, Better Communication

- One message per slide
- Use readable fonts and consistent styles
- Prefer simple diagrams to long paragraphs
- Define technical terms before using jargon
- Make slide titles communicate a point
- Explain a visual; do not read the slide aloud

Choose Your Presentation Improvement Goal

- More eye contact with the entire room
- Speak louder and at a comfortable pace
- Stop reading a phone or full script
- Explain technical ideas with clearer transitions
- Improve visuals and teamwork hand-offs
- Practice: no-phone explanation, audience test, timed rehearsal

Technical Presentations: Tell a Coherent Story

- What problem is the system solving?
- What is technically challenging or innovative?
- How do the components work together?
- Why did you make these design choices?
- What risks remain, and what is the alpha plan?

Resources: MIT: Structuring a Slide Presentation

Make Technical Ideas Visual

- Weak: a long list of React / API / database / ML tools
- Better: a diagram with components, arrows, and labels
- Say what information flows between components
- Explain why the architecture supports your goals
- Use descriptive slide titles, not just “Architecture”

Resources: Cornell Engineering Communication

Presentation 2 — Different from Presentation 1

- Presentation 1: Why this project? Who cares?
- Presentation 2: How will you build the system?
- Show innovations, technical challenges, and feasibility
- Explain architecture and substantive individual responsibilities
- Audience: technical lead or manager who knows the elevator pitch

Presentation 2 — Requirements & Logistics

- Wednesday, November 4 • Maximum 7 minutes/team
- Technical design ~6 min; alpha prototype plan up to ~1 min
- Upload BOTH PPTX and PDF to shared Google Drive before presentation day
- Presentation order: REVERSE alphabetical by team name (Z → A)
- Presentation 1 was A → Z; this time it changes
- All team members should have meaningful speaking roles

Resources: Presentation requirements Official rubric PDF

Suggested Seven-Minute Storyboard

Suggested timing, not an additional rule

- 0:00–0:45 — Project goal and motivation
- 0:45–1:45 — Technical innovations and challenges
- 1:45–3:15 — Architecture and data flow
- 3:15–4:30 — Technology choices and feasibility
- 4:30–5:15 — Component ownership
- 5:15–6:00 — Key risks
- 6:00–7:00 — Alpha prototype plan

Technical Innovation: What Is Difficult?

- What is technically new or particularly challenging?
- Why is a straightforward implementation insufficient?
- Which components or algorithms need investigation?
- What tradeoffs did you evaluate?
- Using an API alone is not necessarily a technical innovation

System Architecture — Show the Big Picture

Example only

- Example: Web UI ↔ Backend API ↔ Scheduling Engine
- Database supports user and assignment information
- Show important modules, relationships, and data flow
- Explain where computation happens and where data is stored
- Your architecture diagram must fit YOUR project

Design Feasibility: Defend Your Choices

- Why did you choose each key framework or technology?
- What role does it play?
- How do components integrate?
- What proof of feasibility exists?
- Which alternatives did you consider?
- Example: justify PostgreSQL using structured data needs

Component Ownership & Workload

- Frontend — responsive interaction and state management
- Backend/API — endpoint and integration design
- Algorithm — constraints and optimization
- Data/testing — validation and evaluation
- Show real technical responsibilities, not just job titles
- Workload should be appropriate for your team size

Technical Risks & Mitigation

- API limits → cache or fallback
- Data quality → validation and tests
- Slow algorithm → benchmark realistic inputs
- Integration blockers → agree interfaces and test early
- Unavailable hardware → simulator or mock input
- Choose 2–3 important risks for YOUR project

Alpha Prototype: What Will Work?

- Define core functionality you plan to demonstrate
- Identify integrated components and testable workflows
- Explain what can realistically be completed by December
- Distinguish alpha scope from the polished final version
- Show concrete progress, not only a wishlist

Rubric 1 — Speaking & Organization

- Four-point scale: Excellent (4), Good (3), Acceptable (2), Poor (1)
- Individual eye contact, body language, and poise
- Individual energy, enthusiasm, and speaking
- Group organization, timing, and visuals
- Excellent: confident delivery and logical flow

Resources: Technical presentation rubric

Rubric 2 — Technical Scope & Content

- Clear system architecture and key components
- Technical innovation, challenges, and scope
- Feasibility and rationale for design choices
- Substantial technical work appropriate to team size
- Technical content carries more weight than speaking skills

Resources: Official technical rubric

Rubric 3 — Alpha Prototype & Checklist

- Is the proposed alpha prototype meaningful?
- Does the plan appear feasible given current progress?
- Can the team grow it into a final system?
- Include diagram, risks, component ownership, and alpha scope
- Rehearse within seven minutes; identify one personal speaking goal

Before Next Wednesday — Final Checklist

- Finalize technical storyline and architecture
- Rehearse 7 minutes as a complete team
- Give everyone a meaningful speaking role
- Upload BOTH .pptx and .pdf files to shared Google Drive
- Open the PDF and check it displays correctly
- Be ready for Z → A reverse alphabetical team order

Teamwork Survey — Due Sunday, November 1

- Submit individually (each student, not one per team)
- Confidential: responses are intended for the instructional team
- Not graded — answer openly and constructively
- Helps us notice uneven workloads, communication issues, and blockers
- Complete it even if your team is working well
- Purpose: find and address team issues early

Demo 0 — Individual Progress Review

- Week of November 3, with assigned instructor
- Show your own GitHub contributions and relevant commits/PRs
- Identify completed and ongoing sprint tickets
- Explain code YOU implemented and important decisions
- Discuss blockers and your next sprint priorities
- Not a formal team presentation; working product not required

Prepare for Demo 0

- Open repository and your branches
- Find meaningful commits and pull requests
- Prepare a short walkthrough of one code contribution
- Update GitHub Projects tickets and sprint board
- Know your challenges and next steps
- Be ready to answer “what does this code do and why?”

Close Out the October Sprint

- Update tickets: done, deferred, or moved forward
- Post weekly updates and document blockers
- Make code contributions traceable
- Review PRs and ensure changes are merged appropriately
- Ask: what did we plan, accomplish, and learn?

Resources: GitHub Projects GitHub Pull Requests

November–December Sprint: Toward Alpha

- Implement key features, not only scaffolding
- Define interfaces and integrate components early
- Test core workflows and investigate high-risk assumptions
- Review responsibilities and revise tickets
- Work toward a meaningful integrated alpha prototype

Hardware / Software Request Reminder

- Only explicitly approved requests may be purchased
- Pending/additional requests need instructor review first
- Keep itemized receipts and supporting documentation
- Follow detailed reimbursement instructions in course email
- Questions about approval? Email your instructor before purchasing

This Week — Key Reminders

- SUN Nov 1: Submit confidential individual teamwork survey
- Before Tue meeting: prepare Demo 0 evidence
- WED Nov 4: Presentation 2, seven minutes/team
- Upload BOTH PPTX and PDF before presentation day
- Teams present in REVERSE alphabetical order (Z → A)
- Keep technical story clear and practice together

References & Learning Resources

- Course site • Presentation requirements • Official rubric
- MIT Communication Lab — structure a presentation
- Cornell Engineering Communication — presenting
- C4 Model — diagram software architecture
- GitHub Projects and pull-request documentation

Resources: Course | Presentation details | Rubric | MIT | Cornell | C4 model